

Instituto Tecnológico de Costa Rica

Curso: CE1101

Introducción a la programación

Documentación técnica

Proyecto: Defensa y Asalto de Base

Estudiantes:

Evan Alejandro Umaña Sojo
Yosep Díaz Marín

Profesores:

Diego Andrés Mora Rojas
Jeff Schmidt Peralta

Semestre I 2026

Contenido

1. Introducción	3
2. Descripción General del Juego	3
3. Reglas del Juego	3
3.1. Objetivo	3
3.2. Orden de una ronda	4
3.3. Economía	4
4. Descripción Específica del Juego	5
4.1. Torres y Estructuras	5
4.2. Unidades del Atacante	6
4.3. Facciones	6
5. Evidencias	7
5.1. Capturas de pantalla	7
5.2. Video de evidencia	8
5.3. Repositorio de GitHub	8
6. Estructuras de Datos Utilizados	8
6.1. Otras estructuras de datos utilizadas	9
7. Sistema de Archivos	9
8. Instrucciones para Ejecutar el Programa	10
9. Crédito de Assets	10
10. Declaración de uso de Inteligencia Artificial y herramientas	10
10.1. Herramientas de IA utilizadas	11
10.2. Descripción del Uso de Inteligencia Artificial	11
10.3. Declaración de Responsabilidad y Autoría	11
11. Conclusiones	12
11.1. Conclusión General	12
11.2. Conclusión Yosep	12
11.3. Conclusión Evan	12
12. Bibliografía	12

1. Introducción

El presente documento corresponde a la documentación técnica y manual de usuario del proyecto Defensa y Asalto de Base, desarrollado como parte del curso de Introducción a la Programación, modalidad Live Learning. A lo largo de este documento se detallan las reglas del juego, las mecánicas de combate y economía, las torres y unidades disponibles, las facciones implementadas, las estructuras de datos utilizadas, la organización del sistema de archivos del proyecto, y las instrucciones necesarias para su ejecución.

2. Descripción General del Juego

Defensa y Asalto de Base es un juego de estrategia para dos jugadores, desarrollado en Python con Tkinter para la interfaz gráfica y pygame para el audio. Un jugador asume el rol de defensor, construyendo muros y torres para proteger su base central; el otro jugador asume el rol de atacante, comprando y posicionando unidades para abrirse paso hasta destruir esa base.

La partida se juega por rondas. En cada ronda, primero el defensor construye su base con el dinero disponible, luego el atacante compra y coloca sus unidades, y finalmente se ejecuta automáticamente la fase de combate, turno por turno, hasta que el atacante destruye la Torre Central o se queda sin unidades vivas. El primer jugador en ganar 3 rondas gana la partida completa.

Antes de jugar, ambos jugadores deben registrarse o iniciar sesión, y cada uno elige una de las tres facciones disponibles, que cambian la apariencia visual de sus torres, muros, unidades y base central.

3. Reglas del Juego

3.1. Objetivo

Ser el primer jugador en ganar 3 rondas. El defensor gana una ronda si logra evitar que el atacante destruya su Torre Central (porque el atacante se quedó sin dinero o

sin unidades vivas), el atacante gana la ronda si logra destruir la Torre Central del defensor.

3.2. Orden de una ronda

1. El defensor recibe su dinero y construye muros y torres dentro de su grid(cuadrícula)
2. El atacante recibe su dinero compra y coloca sus unidades en la zona alrededor del grid del defensor.
3. Se ejecuta automáticamente la fase de combate: cada turno, las torres atacan a las tropas que tengan en rango, y las tropas se mueven y atacan torres, muros o la base.
4. La ronda termina cuando se destruye la Torre Central (gana el atacante) o cuando el atacante se queda sin unidades vivas o sin dinero para seguir comprando (gana el defensor).
5. Se actualiza el marcador de rondas ganadas.
6. Si nadie llegó a 3 rondas ganadas, empieza una ronda nueva.

3.3. Economía

En la primera ronda de la partida, cada jugador arranca con un monto fijo: 500 de dinero para el defensor y 400 para el atacante. De ahí en adelante, el dinero se conserva, y al empezar cada ronda nueva se le suma:

- Un bono fijo de 150 de dinero a cada jugador.
- Para quien va a ser atacante esta ronda: un extra proporcional al daño que le hizo a las estructuras del defensor la última vez que jugó como atacante (la mitad de ese daño, en dinero).

Durante el combate, el defensor gana dinero cada vez que una de sus torres destruye una unidad enemiga, y el atacante gana dinero cada vez que destruye una torre.

4. Descripción Específica del Juego

4.1. Torres y Estructuras

El defensor cuenta con tres tipos de torres ofensivas, además de los muros (que no atacan, solo bloquean el paso) y la Torre Central (la base que hay que proteger).

Todas las torres calculan su rango de ataque contra la tropa enemiga más cercana que tengan a distancia de ataque, y solo pueden tener una habilidad especial activa por tipo de torre.

Torre	Vida	Daño	Rango	Costo	Vel. ataque	Habilidad especial
Cañón	120	10	4	80	1.0	Bala rápida: 15% de probabilidad de disparar con el doble de daño en cada ataque.
Torre Rayo	100	20	3	100	0.8	Cadena de rayo: el ataque puede saltar a una tropa cercana adicional.
Torre Fuego	110	18	3	90	0.9	Quemadura: 25% de probabilidad de prender fuego al objetivo, infligiendo 5 de daño adicional durante 3 turnos.
Muro	80	0	—	10	—	No ataca; solo bloquea el paso y absorbe daño antes de que llegue a las torres o a la base.
Torre Central (la base)	300	0	—	—	—	Es el objetivo final del atacante. Si se destruye, el atacante gana la ronda automáticamente.

4.2. Unidades del Atacante

El atacante cuenta con tres tipos de unidades, cada una pensada para una estrategia distinta: la Básica es barata y se vuelve más fuerte en grupo, el Tanque resiste mucho y es ideal para derribar muros, y el Samurai pega fuerte y se vuelve más letal mientras más enemigos destruye.

Unidad	Vida	Daño	Costo	Vel. ataque	Velocidad	Habilidad especial
Básica	80	15	50	1.0	1.0	Tres es multitud: si se mueve junto a otras Básicas, gana +50% de velocidad y +40% de daño mientras estén juntas.
Tanque	350	20	100	0.3	0.5	Demoledora: el daño contra muros se duplica.
Samurai	120	40	150	0.7	1.5	Espada mágica: cada vez que destruye un objetivo, gana +20% de daño permanente para el resto de la ronda.

4.3. Facciones

Antes de cada partida, ambos jugadores eligen una facción distinta (no pueden repetirse entre el atacante y el defensor). La facción solo cambia la apariencia visual de las torres, muros, unidades y la base central; no afecta directamente las estadísticas de combate.

Facción	Descripción visual
Medieval	Paleta de azules oscuros y dorados, estética inspirada en el juego Clash of Clans.
Jardín Zombie	Paleta de verdes oscuros, estética de naturaleza/zombie inspirada en el juego Plantas vs Zombies.
Robótico	Paleta de morados, azules y negros, estética futurista/tecnológica.

5. Evidencias

5.1. Capturas de pantalla

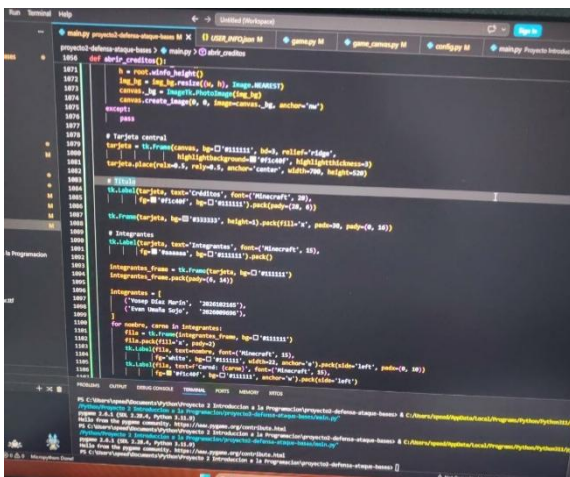


Ilustración 1 Realizando el código

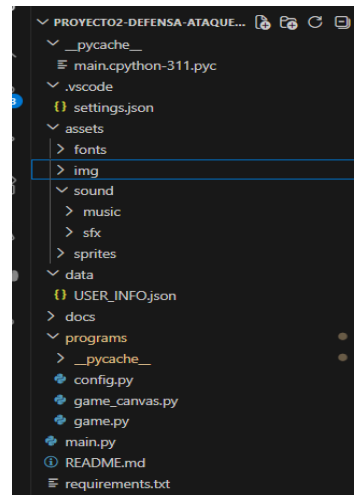


Ilustración 2 Organizando las carpetas

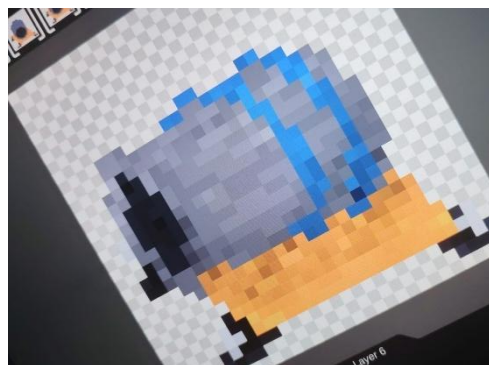
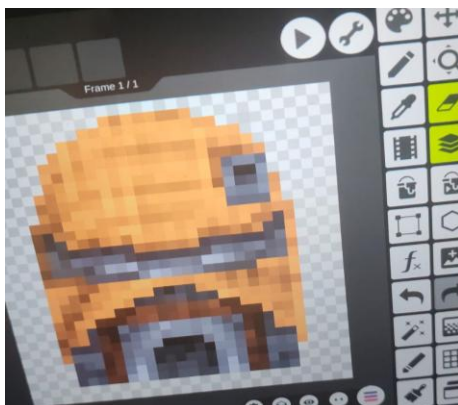


Ilustración 3 y 4 Dibujando y animando los sprites

5.2. Video de evidencia

Link: <https://youtu.be/yZLIRZs4n4c>

5.3. Repositorio de GitHub

Link: <https://github.com/Sarvyz/proyecto2-defensa-ataque-bases/tree/development>

6. Estructuras de Datos Utilizados

El proyecto está organizado en clases, siguiendo el paradigma de programación orientada a objetos. A continuación, se listan las principales clases del archivo `programs/game.py`, que es donde se encuentra toda la lógica del juego:

Clase	Para qué sirve
Pos	Representa una posición (fila, columna) dentro del mapa. Tiene un método para calcular la distancia entre dos posiciones, usado para saber si algo está en rango de ataque.
Objeto	Clase base de la que heredan Estructura y Tropa. Guarda lo que tienen en común: nombre, vida, daño, rango, velocidad de ataque, costo y facción.
Estructura	Hereda de Objeto. Representa muros, torres y la Torre Central. Agrega el atributo <code>es muro</code> para diferenciar muros de torres ofensivas.
Tropa	Hereda de Objeto. Representa las unidades del atacante. Agrega velocidad de movimiento, si su habilidad está activa, turnos de quemadura restantes y bonus de daño acumulado.
GridDefensor	El tablero 10x10 del defensor. Internamente usa un diccionario { Pos: Estructura } para saber qué hay en cada casilla, y métodos para colocar, validar y listar torres/muros vivos.
ZonaAtacante	La zona donde el atacante coloca y mueve sus tropas, en coordenadas "totales" del mapa (incluyen el margen alrededor del grid del defensor).
SistemaCombate	Calcula quién ataca a quién: a qué tropa le dispara cada torre, a qué estructura le pega cada tropa, mueve las tropas y aplica el daño de cada habilidad especial.
Jugador	Representa a un jugador dentro de una partida: nombre, rol actual (atacante/defensor), facción, dinero y rondas ganadas.

EstadoRonda	Controla una ronda completa: las fases (defensor coloca, atacante coloca, combate, fin), y ejecuta los turnos de combate hasta que un bando gana.
Partida	Controla la partida completa: alterna los roles de atacante/defensor entre los dos jugadores cada ronda, administra el dinero entre rondas y determina cuándo alguien gana 3 rondas.

6.1. Otras estructuras de datos utilizadas

- Diccionario { Pos: Estructura/Tropa }: tanto GridDefensor como ZonaAtacante usan un diccionario para guardar qué objeto hay en cada casilla del mapa, lo que permite buscar, agregar o quitar elementos de una celda en tiempo (1).
- Lista de jugadores (data/USER_INFO.json): cada cuenta se guarda como una lista [usuario, contraseña, victorias_atacante, victorias_defensor] dentro de una lista general de cuentas.
- Cola de prioridad (heapq): el algoritmo de pathfinding, usado para mover las tropas del atacante hacia la base evitando muros y torres, usa una cola de prioridad de la librería estándar heapq.

7. Sistema de Archivos

El proyecto guarda la información de los jugadores (usuario, contraseña y victorias) en un archivo data/USER_INFO.json, que se lee al iniciar sesión y se reescribe cada vez que termina una partida (para sumar la victoria correspondiente) o se registra una cuenta nueva. A continuación, se detalla la organización general de carpetas del proyecto:

Archivo / Carpeta	Qué hace
main.py	Archivo principal del programa. Crea la ventana de Tkinter, dibuja el menú, login, registro, cuenta, top de jugadores y selección de facciones. Es el archivo que hay que ejecutar para arrancar el juego.
programs/game.py	Toda la lógica del juego (sin interfaz gráfica): clases de torres, tropas, jugadores, rondas, sistema de combate, pathfinding y economía. Es el "motor" del juego.

programs/game_canvas.py	Interfaz gráfica del tablero de juego: dibuja el grid, las torres, las tropas, los paneles de compra, y maneja los clics del jugador sobre el Canvas de Tkinter.
programs/config.py	Pantalla de configuración (volumen de música y efectos) y todo el manejo de audio con pygame: música de menú, playlist de música de juego, y reproducción de efectos de sonido.
data/USER_INFO.json	Archivo de datos donde se guardan las cuentas de los jugadores: usuario, contraseña y victorias.

8. Instrucciones para Ejecutar el Programa

1. Tener Python 3.11 o superior instalado.
2. Instalar las dependencias del proyecto: `pip install pygame pillow`
3. Ubicarse en la carpeta raíz del proyecto (donde está `main.py`)
4. Ejecutar: `python main.py`
5. Desde el menú principal: registrarse o iniciar sesión con ambos jugadores, elegir "Jugar", elegir quién es el atacante y quién el defensor, y elegir una facción distinta para cada uno.

9. Crédito de Assets

Todos los sprites de torres, unidades, muros y demás elementos visuales del juego fueron dibujados a mano por el equipo, utilizando como referencia imágenes generadas con ChatGPT (a partir de descripciones del estilo visual deseado para cada facción). Las imágenes de fondo fueron hechas con ChatGPT

La música utilizada en el juego se tomó como referencia/fuente del siguiente video (ver bibliografía).

10. Declaración de uso de Inteligencia Artificial y herramientas

Principio de Transparencia Académica

El uso de herramientas de Inteligencia Artificial en proyectos académicos requiere declaración explícita y responsable. Esta sección documenta de forma honesta cómo, cuándo y para qué se utilizaron herramientas de IA en el desarrollo de este proyecto, siguiendo los principios de integridad académica

10.1. Herramientas de IA utilizadas

Herramienta de IA	Propósito
Claude (Anthropic)	Apoyo en la redacción de la documentación técnica, resolución de dudas sobre Tkinter y orientación en la estructura del código.
ChatGPT (OpenAI)	Ayuda en la creación de imágenes y apoyo en la resolución de errores puntuales del código.

10.2. Descripción del Uso de Inteligencia Artificial

Las herramientas de IA se utilizaron como recurso complementario al trabajo propio del equipo, principalmente para aclarar dudas técnicas, obtener sugerencias sobre la estructura del código y apoyar la redacción de la documentación. En todos los casos, el contenido generado fue revisado, comprendido y adaptado por los integrantes del equipo antes de incorporarse al proyecto, sin sustituir en ningún momento el trabajo o el criterio propio.

10.3. Declaración de Responsabilidad y Autoría

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto " Defensa y Asalto de Base" declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

11. **Conclusiones**

11.1. Conclusión General

El desarrollo de Defensa y Asalto de Base permitió aplicar de forma integral los conceptos vistos en el curso: programación orientada a objetos, estructuras de datos, manejo de archivos, algoritmos de búsqueda de caminos y gestión de eventos en una interfaz gráfica. Más allá del resultado final, el proceso de resolver errores reales como problemas de coordenadas, reforzó la importancia de probar cada mecánica antes de darla por funcional.

11.2. Conclusión Yosep

A nivel personal, este proyecto me exigió ir más allá de escribir código que funcione a simple vista tuve que aprender a resolver problemas, entender por qué una torre no hacía daño, aunque pareciera estar bien programada, y rediseñar partes como la economía entre rondas o el manejo de música para que se ajustaran de verdad a lo que pedía el enunciado.

11.3. Conclusión Evan

El desarrollo del proyecto, a pesar de contratiempos y ciertas complicaciones, resultó ser una experiencia enriquecedora para futuros proyectos institucionales y ayudó mucho en la formación como estudiantes

12. **Bibliografía**

[1] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," Python 3.10.14 documentation, 2024.

[2] "Música de referencia (banda sonora del juego)," YouTube, 2026. [En línea]. Disponible en: <https://www.youtube.com/watch?v=oMgQJECVToY>

- [3] Lonely Bunker / 8 Bit Bunker, "Plants vs Zombies Intro Theme Song - Crazy Dave (8-Bit Cover)," YouTube, 2023. [En línea]. Disponible en: <https://www.youtube.com/watch?v=ca59tDL4bRw>
- [4] " 【Clash Royale】 OST 8bit arranged 'Battle theme' (August, 2022 season)," YouTube, 2022. [En línea]. Disponible en: https://www.youtube.com/watch?v=fNz6JZXR_9k
- [5] "Sci-fi 8-bit Chiptune | Background Music for Games | Cyber Factory," YouTube, 2021. [En línea]. Disponible en: <https://www.youtube.com/watch?v=HnE68trDt94>
- [6] Anthropic, "Claude (claude-sonnet-4-6)," Anthropic, San Francisco, CA, USA, 2026. [En línea]. Disponible en: <https://www.anthropic.com>
- [7] OpenAI, "ChatGPT (GPT-4o)," OpenAI, San Francisco, CA, USA, 2026. [En línea]. Disponible en: <https://www.openai.com>